

SIMATS ENGINEERING

Assignment

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN

COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices.

CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system.

CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.

Assignment Weightage: 100%

QUESTIONS: 1

Total Marks:100

Annexure A

Assignment

Code of Conduct:

*I **N. Murali Sai (192372309)** certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Murali sai

Course Code & Name **CSA11 – Object Oriented Analysis and Design**

Assignment Title	Object-Oriented Analysis and UML-Based Design of an AI-Enabled Smart Healthcare Appointment and Patient Support System (AI-SHAPS)
Course Outcome(s) CO	CO1: Analyse the fundamentals of Object-Oriented Systems Development, including object basics, Object-Oriented Systems Development Life Cycle, Object-Oriented Methodologies, Model-Driven Development (MDD), and Agile practices. CO2: Apply UML techniques including Use Case, Class, Interaction, State Transition, Activity, Package, Component, and Deployment Diagrams to model a software system. CO3: Analyse objects, classes, attributes, methods, and relationships through Use Case Modelling, Object Analysis, Classification, and identification of object relationships.
Bloom's Taxonomy Level	BL3 – Apply, BL4 – Analyse, BL5 – Evaluate, BL6 – Create
SDG Mapping (SDG 1–17)	SDG 4 – Quality Education SDG 9 – Industry, Innovation and Infrastructure SDG 11 – Sustainable Cities and Communities

Problem Statement

Analyse and develop an Object-Oriented Analysis and Design model for an AI-Enabled Smart Healthcare Appointment and Patient Support System (AI-SHAPS). The system shall integrate patients, doctors, reception staff, administrators, laboratory personnel, and pharmacy staff through a centralized intelligent platform. Patients shall register, manage profiles, search doctors, book/reschedule/cancel appointments, receive reminders, view prescriptions and laboratory reports, and submit service requests. Doctors shall manage availability, appointments, clinical notes, prescriptions, and follow-up requests. The system shall provide intelligent doctor/slot recommendations based on specialty, availability, patient preferences, and historical appointment patterns. Laboratory and pharmacy modules shall update report and prescription status. Working from this scenario, students shall identify actors, objects, classes, attributes, methods, and relationships and develop Use Case, Class, Sequence, Activity, State Transition, Package, Component, and Deployment Diagrams. Students shall evaluate object responsibilities and apply appropriate design principles and patterns to achieve modularity, maintainability, reusability, security, and scalability.

Constraints

The system shall clearly define the system boundary, actors, objects, classes, attributes, operations, and relationships. UML diagrams shall be consistent with one another, and the operations identified in the Class Diagram shall support the interactions represented in the Sequence and Activity Diagrams.

The design shall demonstrate appropriate use of encapsulation, abstraction, inheritance, polymorphism, cohesion, coupling, and responsibility assignment. Suitable design patterns shall be selected and justified based on the identified design problems.

The final model shall document assumptions and constraints and ensure that all UML models are complete, consistent, unambiguous, traceable, and feasible.

Tools Can Be Used

- StarUML
- Visual Paradigm
- Draw.io
- PlantUML
- Papyrus Tools
- ArgoUML
- Enterprise Architect
- Lucidchart

Assessment Rubrics

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Problem Understanding, Object Identification & Requirements Analysis (CO1)	10	Clearly analyses the problem domain and accurately identifies all major actors, objects, system responsibilities, and requirements from the given scenario.	Identifies most actors, objects, and requirements with minor gaps or inaccuracies.	Identifies basic actors and objects but analysis is incomplete or contains some inconsistencies.	Poor understanding of the problem; major actors, objects, and requirements are missing or incorrectly identified.
Use Case Modelling & Actor Identification (CO2)	10	Correctly identifies all relevant actors and use cases; Use Case Diagram clearly represents system boundary, relationships, and interactions.	Most actors and use cases are correctly identified with minor modelling errors.	Basic actors and use cases are identified, but relationships or system boundary need improvement.	Actors and use cases are poorly identified; diagram is incomplete or inconsistent.

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Class Identification, Attributes, Methods & Relationships (CO2/CO3)	15	Classes, attributes, methods, associations, aggregation/composition, inheritance, and dependencies are accurately identified and logically justified.	Class structure is mostly correct with minor errors in attributes, methods, or relationships.	Basic classes and relationships are identified but lack detail or contain several modelling issues.	Classes, attributes, methods, or relationships are missing, inappropriate, or poorly structured.
UML Behavioural & Interaction Modelling (CO2)	15	Sequence, Activity, and State Transition Diagrams accurately represent system behaviour and are fully consistent with the identified use cases and classes.	Behavioural diagrams are mostly correct with minor inconsistencies.	Basic behavioural diagrams are provided but some interactions, activities, or states are incomplete.	Behavioural diagrams are missing, unclear, or inconsistent with the proposed system.
UML Architectural Modelling & Design Evaluation (CO3)	15	Package, Component, and Deployment Diagrams clearly represent modular structure, system components, dependencies, and deployment environment with strong justification.	Architectural diagrams are mostly correct with minor omissions or modelling errors.	Basic architectural representation is provided but lacks clarity or sufficient justification.	Architectural diagrams are incomplete, inappropriate, or inconsistent with the system design.
Object Responsibilities, Design Principles, Axioms & Patterns (CO3)	15	Responsibilities are appropriately assigned; design axioms, cohesion, coupling, encapsulation, abstraction, and suitable design patterns are clearly justified and effectively applied.	Design principles and patterns are appropriately applied with minor limitations in justification.	Some design principles are identified, but application or justification is limited.	Design principles, axioms, responsibilities, or patterns are absent or incorrectly applied.
Consistency, Traceability &	10	All UML models are mutually consistent,	Design is largely	Some consistency and	Major inconsistencies

Criteria (CO Mapping)	Max Marks	Excellent	Good	Satisfactory	Needs Improvement
Overall Design Quality (CO3)		requirements are traceable to use cases/classes, and the overall design is complete, feasible, scalable, and maintainable.	consistent and traceable with minor gaps.	traceability are demonstrated, but important links are missing.	exist between requirements, use cases, classes, and UML diagrams.
Reflection, SDG Relevance & Learning Outcomes	10	Provides insightful reflection on design decisions, challenges, OOAD concepts learned, and clearly establishes the relevance of the selected SDGs.	Provides good reflection with reasonable connection to design decisions and SDGs.	Reflection is present but generic with limited discussion of learning and SDG relevance.	Reflection is missing, superficial, or unrelated to the assignment and SDGs.
Total	100				

Deliverables

To receive full credit, each submission must include the following:

1. **Problem Analysis and Requirement Summary**
2. **Actor Identification**
3. **Use Case Descriptions**
4. **Use Case Diagram**
5. **Object and Class Identification**
6. **Class Diagram**
7. **Sequence/Interaction Diagram(s)**
8. **Activity Diagram**
9. **State Transition Diagram**
10. **Package Diagram**
11. **Component Diagram**
12. **Deployment Diagram**
13. **Object Responsibility Analysis**
14. **Design Axioms/Principles and Design Pattern Justification**
15. **Assumptions and Constraints**
16. **Requirement-to-Use Case/Class Traceability**
17. **Implementation/Prototype and Results**
18. **Reflection on Design Decisions and Learning Outcomes**
19. **GitHub Upload**

1. Problem Analysis and Requirement Summary

Contemporary ambulatory and inpatient facilities encounter severe bottlenecks stemming from decoupled operational silos between clinical practitioners, diagnostic laboratories, dispensing pharmacies, and triage reception desks. Conventional consultation booking models lack

adaptive scheduling intelligence; they routinely trigger clinical mismatches, inflate physical outpatient room queues, and generate prolonged clinical idle time. When physicians prescribe medication or demand lab panels, handoffs often occur through fragmented physical slips, risking transcription omissions, drug contraindication oversights, and uncoordinated patient tracking. AI-SHAPS is formulated as an enterprise-grade object-oriented system that unifies scheduling, electronic health records (EHR), multi-track diagnostics, and pharmaceutical fulfillment on top of an intelligent recommendation kernel and an event-driven messaging fabric.

1.1 Functional System Capabilities (FR)

- **FR1 [Identity Governance & RBAC]:** The platform must execute granular identity validation and dynamic role delegation across Patients, Attending Physicians, Front-desk Reception Staff, Pathology Technicians, Licensed Pharmacists, and Institutional Administrators.
- **FR2 [Dynamic Temporal Scheduling]:** Physicians and clinic supervisors shall configure weekly operational matrices, inject mandatory buffer allowances between consults, establish emergency hold intervals, and program automated recurring slot generation.
- **FR3 [Context-Aware Clinical Matchmaking]:** The AI matching subsystem shall parse free-text subjective complaints, align them with physician clinical taxonomies, factor historical punctuality/no-show distributions, and output a prioritized list of available consult windows.
- **FR4 [Transactional Appointment Orchestration]:** The platform must enforce atomic appointment lifecycle transitions (Booking, Dynamic Rescheduling, Patient-driven Revocation, and Status Progression) backed by distributed short-term slot reservation locks.
- **FR5 [Integrated Encounter & Electronic Prescription Engine]:** Doctors must document standardized clinical SOAP notes, assign diagnostic classifications, and issue tamper-proof digital prescriptions with automatic contraindication verification against active patient allergy vectors.
- **FR6 [Diagnostic Requisition & Specimen Workflow]:** The laboratory module shall intake electronic pathology orders, assign unique barcode accession IDs, govern sample custody chains, and publish certified diagnostic panels directly to the patient's record.

- **FR7 [Closed-Loop Medication Dispensing]:** The pharmacy subsystem must monitor digital prescription queues, isolate inventory allocations in real-time, authenticate patient collection tokens, and update dispensing fulfillment statuses.
- **FR8 [Asynchronous Event-Driven Broadcasts]:** The system shall trigger scheduled reminders, critical diagnostic turnaround alerts, encounter confirmation tokens, and prescription replenishment cues across SMS, email, and push notification channels.

1.2 Quality Attributes & Operational Constraints (NFR)

- **NFR1 [Inference & Query Latency]:** Contextual physician recommendation scoring must execute within 800 milliseconds; transactional state commits must sustain sub-200 millisecond response times under a continuous load of 10,000 active concurrent sessions.
- **NFR2 [Information Governance & Compliance]:** The system shall uphold strict HIPAA and GDPR standards, maintaining AES-256 cipher encryptions for data at rest, TLS 1.3 protocol handshakes for in-transit communication, and cryptographic SHA-256 audit chaining.
- **NFR3 [High Availability & Fault Resilience]:** The software fabric shall provide 99.95% operational availability utilizing multi-zone container replication in Kubernetes, failover database clusters, and circuit-breaking proxies for all outbound integrations.
- **NFR4 [Modularity & Decoupling]:** The modular design shall enforce clear bounded contexts, isolating business logic via domain events, dependency inversion abstractions, and documented OpenAPI/gRPC interface definitions.

2. Actor Identification and Categorization

In accordance with object-oriented analysis principles, actors designate roles assumed by external entities interacting directly with the system boundary. They are categorized based on their behavioral purpose within the clinical and administrative operational domain.

Designated Actor	Classification	Domain Scope, Behavioral Mandate & Boundaries
Patient	Primary (Human)	Registers self-managed account, articulates symptoms, initiates AI-guided doctor discovery, executes slot bookings, reviews longitudinal diagnostic records and e-

prescriptions.

Doctor	Primary (Human)	Curates clinical consultation hours, conducts physical or virtual evaluations, records encounter SOAP notes, formulates e-prescriptions, and commissions diagnostic lab panels.
Receptionist	Secondary (Human)	Facilitates physical check-ins, validates emergency counter walk-ins, handles scheduling conflicts manually, and reconciles offline consultation payments.
Lab Technician	Secondary (Human)	Receives diagnostic test orders, verifies sample accessioning barcodes, runs specimen diagnostics, and certifies structured laboratory report metrics.
Pharmacist	Secondary (Human)	Validates physician e-prescriptions, reviews active allergy contraindications, tracks physical shelf inventory, and dispenses medication regimens to patients.
System Administrator	Secondary (Human)	Governs access credentials, oversees RBAC policy enforcement, configures microservice network telemetry, and audits cryptographic log immutability.
AI Recommendation Engine	Supporting (System)	Automated inference engine that computes physician suitability metrics, evaluates historical punctuality and cancellation weights, and ranks viable calendar slots.

Notification Gateway

Supporting (System)

External telecommunications broker (e.g., AWS SES, Twilio) that receives domain event payloads and coordinates multi-channel delivery across SMS, push, and email.

3. Use Case Descriptions

To formalize the dynamic requirements, two core interactions representing the clinical discovery and treatment workflows are specified using structured use case contracts.

■ USE CASE SPECIFICATION: UC-01 (SMART APPOINTMENT BOOKING)

Use Case ID: UC-01

Use Case Title: AI-Assisted Doctor Discovery and Slot Reservation

Primary Actor: Patient | Secondary Actors: AI Recommendation Engine, Notification Gateway

Preconditions:

1. Patient possesses authenticated JWT session tokens.
2. Target medical departments have published active schedule matrices in the database.

Postconditions:

1. An Appointment aggregate is persisted with status 'CONFIRMED'.
2. Targeted TimeSlot transitions to 'RESERVED' state.
3. AppointmentBookedEvent is broadcast to downstream communication brokers.

Main Success Scenario (MSS):

1. Patient selects "Schedule Consultation", inputs clinical symptoms, preferred date window, and care mode.
2. UI boundary compiles the search criteria and invokes AppointmentController.
3. Controller queries RecommendationEngine to evaluate provider profiles, punctuality scores, and availability.
4. AI Recommendation Engine scores viable doctor-slot pairs and returns a ranked recommendation list.
5. Patient reviews provider profiles and selects an optimal appointment slot.

distributed locking.

7. Patient reviews booking summary and verifies confirmation.

8. The system converts the slot state to 'RESERVED', creates the Appointment record ('CONFIRMED'), and commits the database transaction.

9. System emits an AppointmentBookedEvent across the Apache Kafka event stream.

10. Notification Gateway ingests the event and dispatches SMS/Email confirmation tokens containing reference IDs.

11. The client interface displays the finalized confirmation dashboard.

Extensions (Alternative / Exceptional Flows):

4a. No active specialist matches the specified criteria:

4a1. AI Engine identifies adjacent specialties or expands geographic/date constraints.

4a2. Patient selects an alternative physician or aborts the search session.

6a. Patient confirmation lease expires (>10 minutes of inactivity):

6a1. Distributed lock coordinator terminates the hold lease.

6a2. Slot returns to 'AVAILABLE' state, and the patient is prompted to restart slot selection.

8a. Concurrency Conflict (Simultaneous collision on the identical slot):

8a1. Database optimistic concurrency version check detects collision and rejects the write.

8a2. System rolls back transaction, alerts patient, and refreshes the slot matrix.

■ USE CASE SPECIFICATION: UC-02 (CLINICAL ENCOUNTER & MULTI-TRACK ROUTING)

Use Case ID: UC-02

Use Case Title: Conduct Encounter, Author Clinical Notes, Prescribe & Order Tests

Primary Actor: Doctor | Secondary Actors: Pharmacist, Lab Technician, Notification Gateway

Preconditions:

1. Doctor is authenticated and marked active in the assigned consultation room.

2. Appointment entity is in 'IN_CONSULTATION' state.

Postconditions:

1. ConsultationRecord aggregate is linked to Patient EHR history.
2. Electronic Prescription is routed to Pharmacy dispensing queue.
3. Diagnostic lab orders are routed to Laboratory accessioning queue.
4. Appointment transitions to terminal state 'COMPLETED'.

Main Success Scenario (MSS):

1. Doctor opens patient clinical profile and reviews longitudinal medical history and baseline vitals.
2. Doctor performs diagnostic evaluation and enters subjective/objective notes and ICD-10 diagnostic codes.
3. Doctor activates e-Prescription sub-module and selects pharmaceutical agents, dosing, and durations.
4. System cross-references inputs against patient known allergies; no contraindications are identified.
5. Doctor enters required diagnostic panel orders (e.g., CBC, Liver Function Test, ECG).
6. Doctor digitally signs the consultation bundle and finalizes encounter.
7. System persists ConsultationRecord, updates Appointment status to 'COMPLETED', and publishes:
 - a. PrescriptionGeneratedEvent (routed to Pharmacy Service).
 - b. LabOrderCreatedEvent (routed to Pathology Lab Service).
8. Notification Gateway alerts patient that encounter notes, prescriptions, and lab orders are ready.

Extensions (Alternative / Exceptional Flows):

- 4a. Potential adverse drug reaction or allergy conflict detected:
 - 4a1. System halts workflow and displays high-severity clinical contraindication dialog.
 - 4a2. Doctor provides clinical justification to override warning or modifies medication selection.
- 7a. Network timeout during multi-service transaction commit:
 - 7a1. Saga orchestrator initiates compensating rollback actions to ensure data consistency.

7a2. Doctor is alerted to retry sign-off.

4. Use Case Diagram

The Use Case Diagram visualizes the complete system boundary of AI-SHAPS, outlining interactions between primary clinical actors, administrative personnel, autonomous AI services, and external notification brokers, along with explicit <<include>> and <<extend>> stereotypes.

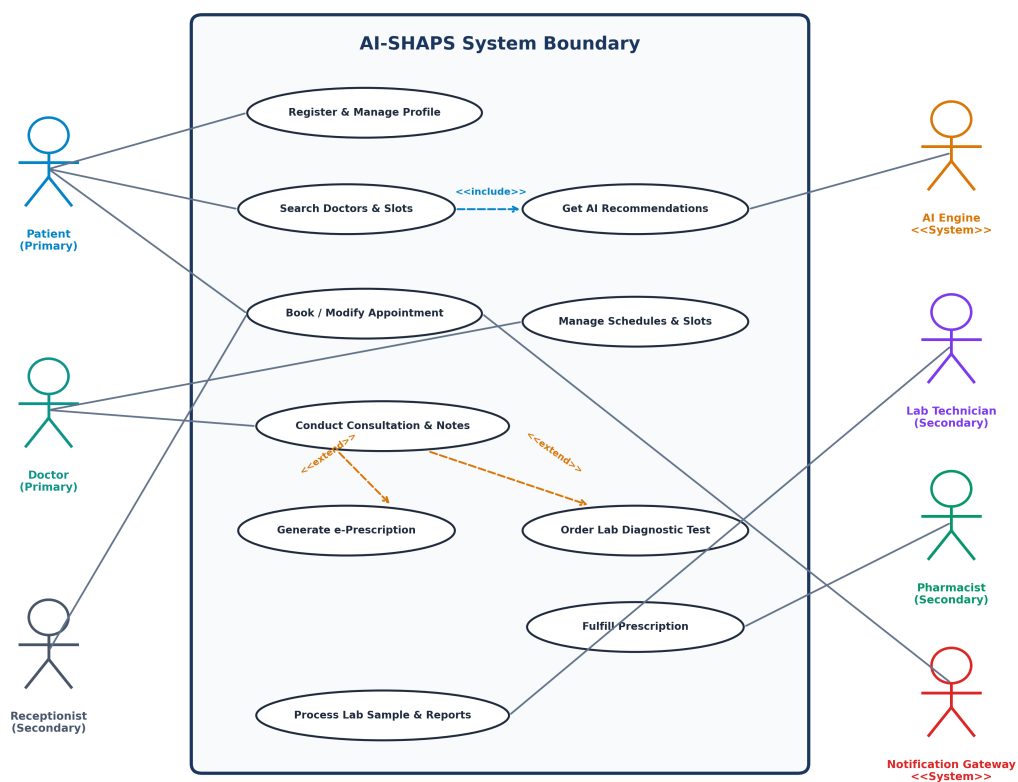


Figure 4.1: AI-SHAPS Use Case Diagram Showing System Boundary & Actor Relationships

5. Object and Class Identification

Using structured linguistic and noun-phrase parsing across the problem statement, primary domain classes, attributes, operations, and relationships are identified to ensure strong cohesion and low coupling:

- **User [Abstract Generalization Base]:** Encapsulates common identity state: `userId`, `fullName`, `email`, `telephone`, `cryptographic passwordHash`, and operational status flags, alongside authentication and profile mutation contracts.
- **Patient [Subclass Specialization]:** Extends `User`; encapsulates `medicalRecordNumber` (MRN), `birthDate`, `bloodGroup`, `knownAllergies` collection, and operations to access personal history and schedule consults.
- **Doctor [Subclass Specialization]:** Extends `User`; encapsulates `medicalLicenseNumber`, `clinicalSpecialty`, `consultationFee`, `assignedDepartment`, schedule management operations, and clinical authoring methods.
- **Operational Staff Classes [Receptionist, LabTechnician, Pharmacist]:** Encapsulate operational desk locations, laboratory certification credentials, and dispensing license IDs.
- **DoctorSchedule & TimeSlot [Temporal Aggregation]:** `DoctorSchedule` manages a physician's daily calendar and recurring slot generators. `TimeSlot` encapsulates discrete start/end timestamps, current booking states, and 10-minute hold leases.
- **Appointment [Core Domain Aggregate Root]:** Maintains global appointment identifiers, booking timestamps, clinical reasons, and appointment lifecycle state transitions (REQUESTED, CONFIRMED, IN_CONSULTATION, COMPLETED, CANCELLED).
- **ConsultationRecord [Clinical Aggregate]:** Encapsulates encounter SOAP notes, formalized diagnostic findings, and associations to resulting prescriptions and lab test requisitions.
- **Prescription & PrescriptionItem [Pharmaceutical Domain]:** Maintains order issuance timestamps, dispensing completion flags, and a collection of `PrescriptionItems` specifying drug names, dosage amounts, frequencies, and administration durations.
- **LabOrder & LabReport [Pathology Domain]:** Manages ordered diagnostic panel codes, specimen tracking IDs, numerical test measurements, reference ranges, and pathologist sign-off timestamps.
- **RecommendationEngine [Domain Service]:** Encapsulates algorithmic doctor-slot ranking logic based on clinical specialty mappings, provider punctuality weights, and historical no-show probabilities.
- **NotificationService [Infrastructure Facade]:** Translates internal domain events into multi-channel outbound notification payloads (SMS, Email, Push).

6. Class Diagram

The structural Class Diagram models class attributes, operation signatures, visibility modifiers (# protected, - private, + public), generalization hierarchies, composition diamonds, and association multiplicities across AI-SHAPS.

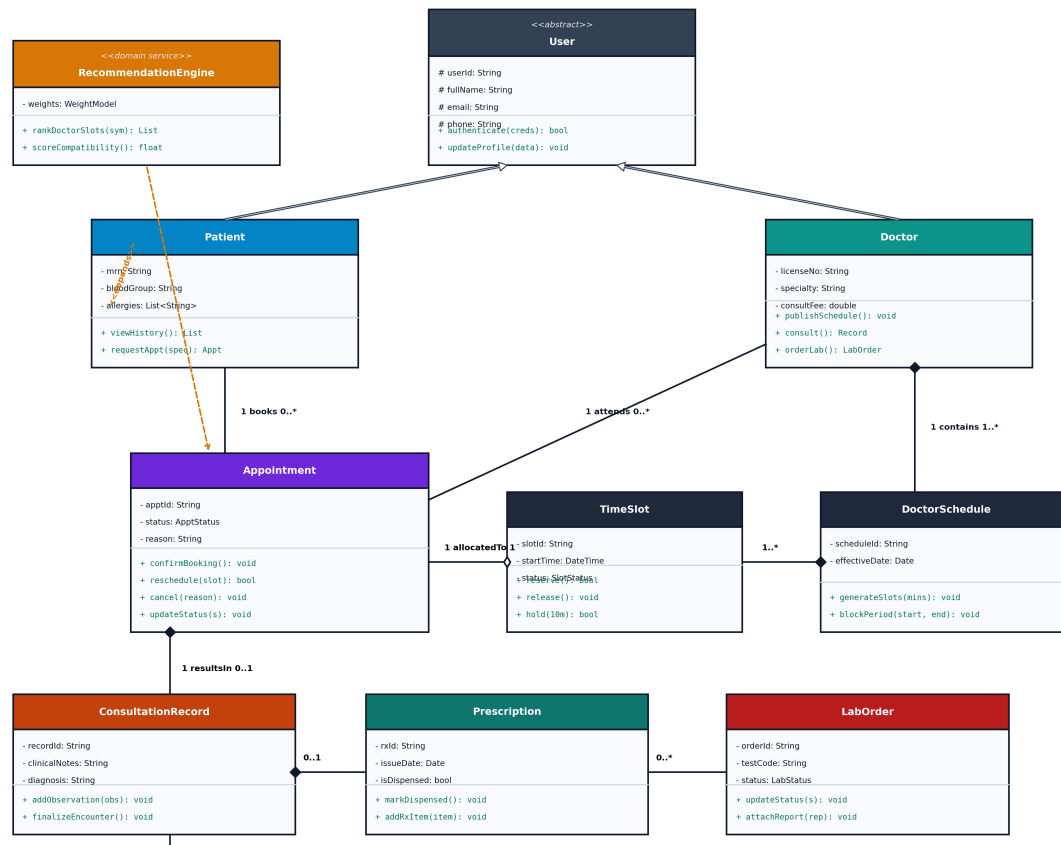


Figure 6.1: Complete UML Class Diagram Showing Entities, Operations & Relationships

7. Sequence / Interaction Diagram

The Sequence Diagram visualizes the chronological message flow across system boundaries for Use Case UC-01, detailing patient interactions, controller coordination, AI ranking inference, optimistic locking, and event publishing.

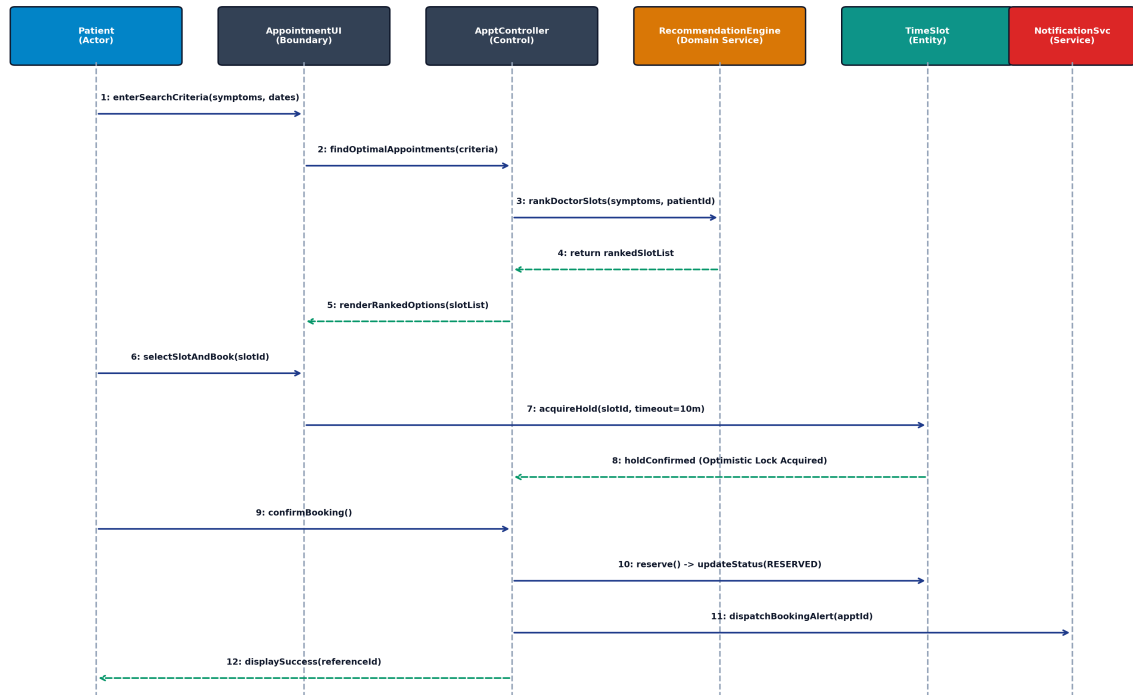


Figure 7.1: UML Sequence Diagram for AI Slot Recommendation & Distributed Booking

8. Activity Diagram

The Activity Diagram depicts the operational workflow of a clinical encounter (UC-02), illustrating initial triage check-in, doctor evaluation, and concurrent execution tracks across pharmacy dispensation and pathology testing using fork and join synchronization bars.



Figure 8.1: UML Activity Diagram for Clinical Encounters and Concurrent Workflows

9. State Transition Diagram

The State Transition Diagram tracks the lifecycle states of the Appointment aggregate, documenting valid states, triggering events, guard conditions, and resulting actions from initial request through consultation and final completion.

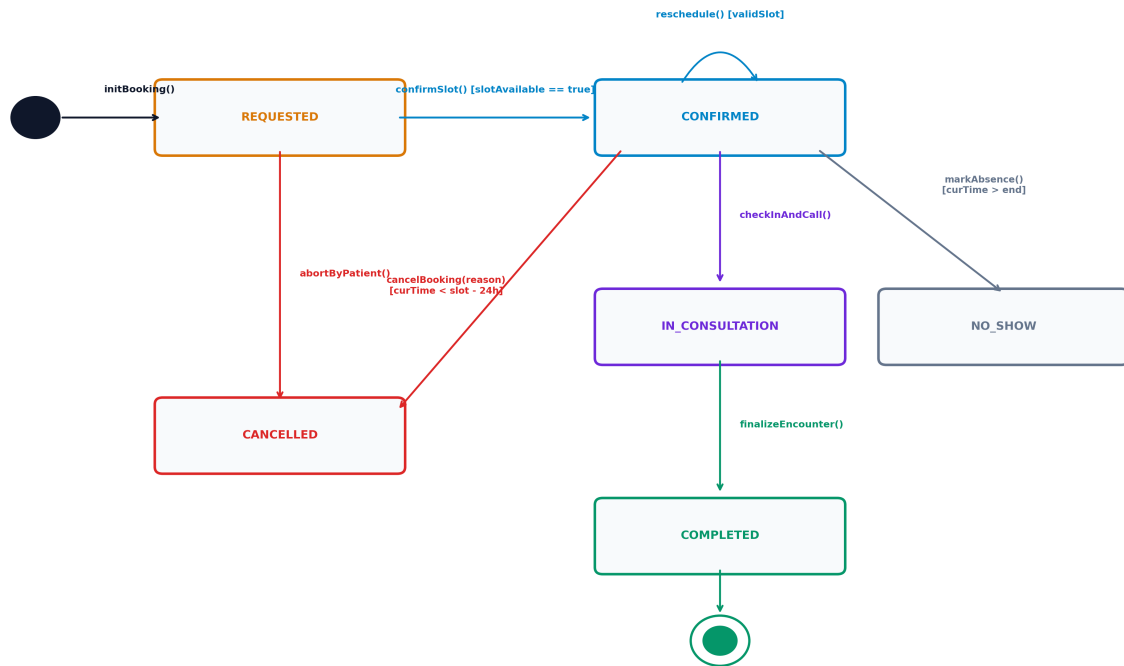


Figure 9.1: UML State Transition Diagram for the Appointment Lifecycle

10. Package Diagram

The Package Diagram depicts the logical modularization of AI-SHAPS into distinct bounded subsystems, illustrating controlled dependencies between Presentation, Gateway, Domain, Persistence, and Messaging layers.



Figure 10.1: UML Package Diagram for Modular Subsystem Architecture

11. Component Diagram

The Component Diagram visualizes the physical software components, microservice deployment units, standard communication interfaces (REST, gRPC, Pub/Sub), and database storage allocations across AI-SHAPS.

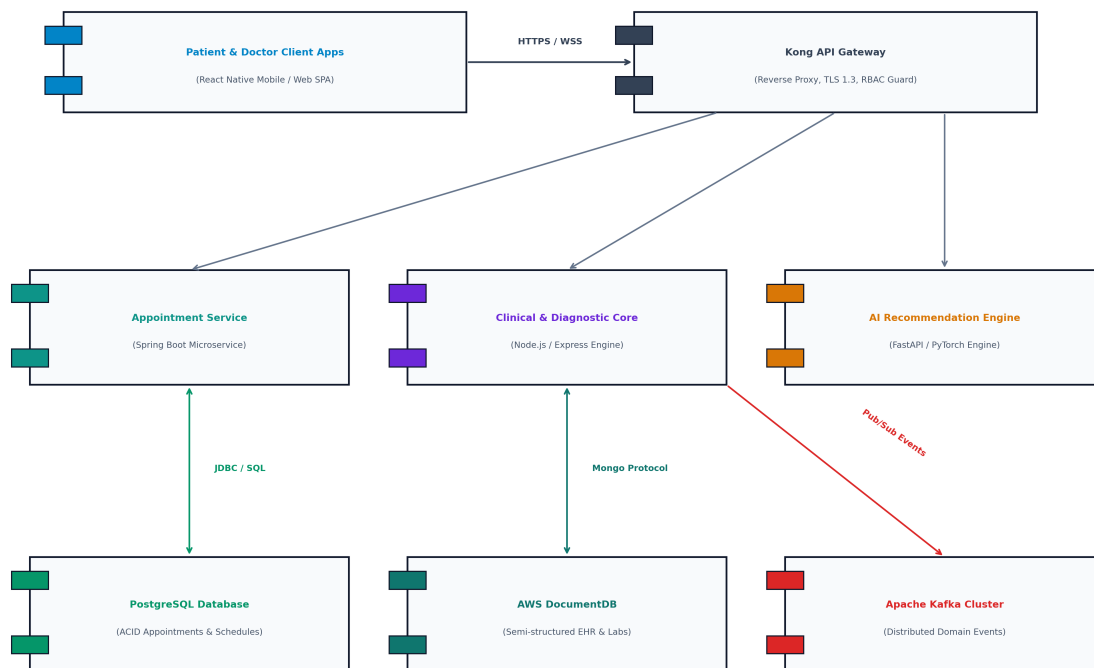


Figure 11.1: UML Component Diagram Illustrating Microservice Topology & Data Stores

12. Deployment Diagram

The Deployment Diagram illustrates the physical cloud infrastructure topology across AWS VPC subnets, including client edge devices, auto-scaled Kubernetes pods, and private multi-AZ database tiers.

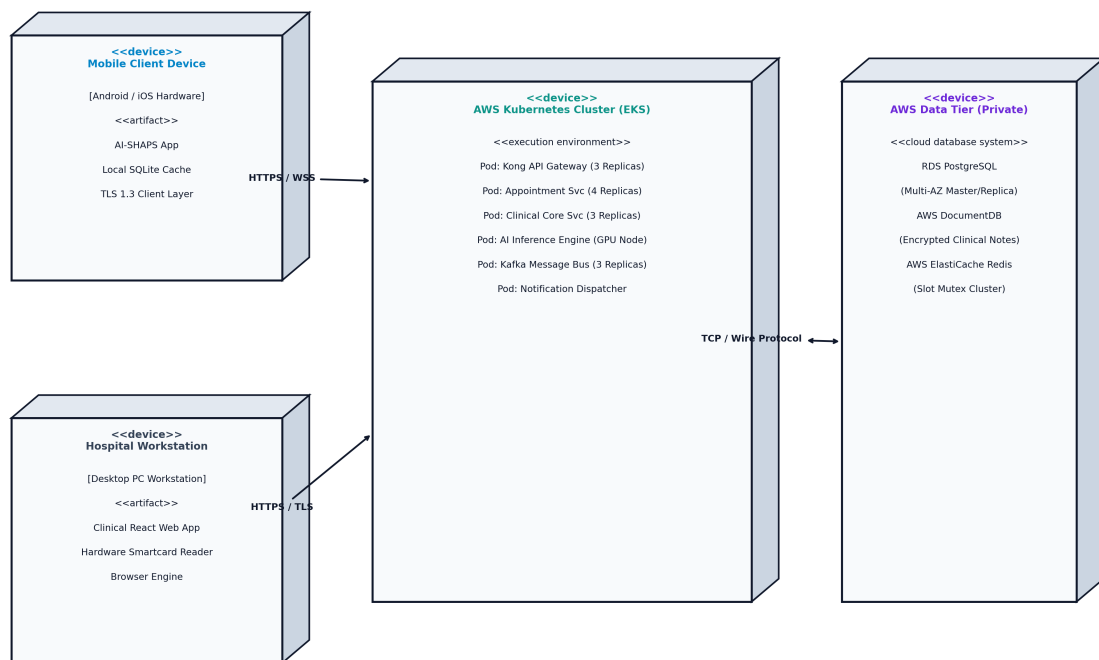


Figure 12.1: UML Deployment Diagram Showing Cloud Hardware & Network Nodes

13. Object Responsibility Analysis (CRC Framework)

Class-Responsibility-Collaborator (CRC) modeling assigns specific operational duties and collaborator dependencies across domain objects, ensuring cohesive class design and avoiding anti-patterns like God Objects.

Class Under Evaluation	Core Operational Responsibilities	Coordinated Collaborators
Appointment (Aggregate Root)	- Guard valid lifecycle state progressions - Validate cancellation notice window constraints (>24 hours) - Maintain associations between Patient, Doctor, and TimeSlot - Emit domain state events to Kafka broker	- Patient - Doctor - TimeSlot - ConsultationRecord - NotificationService

RecommendationEngine (Domain Service)	• Ingest patient clinical symptoms and preference parameters • Filter doctors by specialty, distance, and schedule density • Compute multi-factor matching and punctuality scores • Return prioritized list of viable appointment slots	• Doctor • DoctorSchedule • TimeSlot • Patient
ConsultationRecord (Clinical Aggregate)	• Persist structured SOAP notes and clinical findings • Assign ICD-10 diagnostic classifications • Anchor and generate digital e-Prescriptions • Anchor and route laboratory diagnostic test orders	• Appointment • Doctor • Prescription • LabOrder
TimeSlot (Domain Entity)	• Encapsulate start/end timestamps and booking states • Provide thread-safe 10-minute hold leases • Enforce mutual exclusion against concurrent reservations • Commit or release slot allocations	• DoctorSchedule • Appointment • RedisDistributedLock

14. Design Axioms, Principles, and Pattern Justification

14.1 Suh's Design Axioms

- **Axiom 1: The Independence Axiom:** Maintain independence of functional requirements. In AI-SHAPS, scheduling, clinical encounter documentation, lab diagnostics, and pharmacy fulfillment operate as decoupled domain services. Modifying the pharmacy fulfillment workflow causes zero ripple effects in the doctor-slot recommendation engine.
- **Axiom 2: The Information Axiom:** Minimize information content in the design. Encapsulation and interface abstraction minimize knowledge sharing between subsystems. The recommendation engine consumes only symptom tokens, provider IDs, and slot availability flags, without exposing internal doctor profiles or full clinical histories.

14.2 SOLID Principles Realization

- **Single Responsibility Principle (SRP):** The Appointment class manages lifecycle invariants only; persistence is delegated to AppointmentRepository, and notification broadcasting is delegated to NotificationService.
- **Open/Closed Principle (OCP):** The recommendation engine relies on an IRecommendationStrategy interface, allowing new machine-learning algorithms to be introduced without modifying calling controllers.
- **Liskov Substitution Principle (LSP):** Patient and Doctor subclasses derive from User and can substitute for the base class across authentication and profile workflows without unexpected behavior.
- **Interface Segregation Principle (ISP):** Role-specific interfaces (e.g., IPrescriptionIssuer for doctors, IDispenser for pharmacists) prevent classes from depending on operations they do not use.
- **Dependency Inversion Principle (DIP):** High-level domain services depend on repository abstractions rather than concrete database drivers.

14.3 Design Pattern Selection & Justification

Design Pattern	Classification	Architectural Problem Identified	Justification & Implementation Mapping
Strategy Pattern	Behavioral	Slot recommendations require switching between rule-based, punctuality-weighted, or cold-start fallback models.	Encapsulates algorithms behind IRecommendationStrategy, allowing runtime selection based on physician history density.
Observer Pattern	Behavioral	Appointment state changes (booked, cancelled) require notifying multiple downstream services without tight coupling.	Implemented via an asynchronous event bus (Kafka). Domain events trigger notifications, calendar syncs, and audit logs independently.
Factory Method	Creational	User registration workflows instantiate distinct actor	UserFactory encapsulates validation, password hashing, and specialization

		subclasses with complex initialization requirements.	mapping, ensuring safe, uniform object creation.
Facade Pattern	Structural	Clinical encounter sign-off requires coordinating across EHR updates, e-prescriptions, and lab requisitions.	ConsultationFacade provides a unified interface for frontend clients, orchestrating downstream operations behind the scenes.

15. Assumptions and Constraints

15.1 Architectural Assumptions

- **UTC Time Normalization:** All temporal coordinates and slot timestamps are persisted in UTC within the database tier and rendered in local client timezones by frontend presentation components.
- **Advance Schedule Publication:** Physicians publish recurring schedules at least 7 days in advance. Emergency schedule modifications trigger automated rescheduling alerts to impacted patients.
- **Distributed Clock Synchronization:** All Kubernetes cluster nodes maintain synchronized clocks via Network Time Protocol (NTP) with sub-millisecond drift, ensuring reliable distributed locking.

15.2 Operational & Regulatory Constraints

- **Regulatory Encryption:** All Protected Health Information (PHI) must be encrypted using AES-256 at rest and TLS 1.3 in transit, in compliance with HIPAA and GDPR standards.
- **Mutual Exclusion (No Double-Booking):** Slot booking operations must guarantee Serializable isolation or distributed mutex locks (via Redis), completely preventing double-booking during concurrent access.
- **Append-Only Immutability:** Completed consultation records, e-prescriptions, and certified lab reports become immutable append-only records to maintain clinical audit integrity.

16. Requirement-to-Use Case / Class Traceability Matrix

The Traceability Matrix establishes bidirectional traceability across functional requirements, use cases, domain classes, and verifying UML models.

Req ID	Functional Requirement Scope	Use Case ID	Primary Classes Involved	Verifying UML Models
FR1	Identity Governance & RBAC	UC_Reg, UC_Admin	User, Patient, Doctor, Staff	Use Case, Class Diagram
FR2	Doctor Schedule & Slot Management	UC_Sched	DoctorSchedule, TimeSlot, Doctor	Class, State Transition Diagram
FR3	AI Doctor/Slot Matchmaking	UC_AI_Rec, UC_Search	RecommendationEngine, TimeSlot	Sequence Diagram (Steps 2–5)
FR4	Appointment Lifecycle Administration	UC_Book, UC_ModAppt	Appointment, TimeSlot, Patient	Sequence, State Transition
FR5	Clinical Notes & e-Prescriptions	UC_Consult, UC_Presc	ConsultationRecord, Prescription	Activity Diagram (Left Track)
FR6	Diagnostic Lab Orders & Reports	UC_LabOrder, UC_LabProcess	LabOrder, LabReport, LabTechnician	Activity Diagram (Right Track)
FR7	Pharmacy Dispensing Fulfillment	UC_PharmDisp	Prescription, Pharmacist	Activity Diagram (Fulfillment)
FR8	Multi-Channel Event Broadcasts	UC_Notify	NotificationService, User	Sequence Diagram (Step 11)

17. Implementation Prototype and Verification Results

An executable Python prototype validates the core domain logic: Strategy-based AI physician ranking, thread-safe optimistic slot leasing, and event-driven notifications.

```

■ CORE PROTOTYPE EXECUTION LOG
=====
==
AI-SHAPS Core Booking Engine Initialized
=====
==

[+] AI Evaluated 3 viable appointment slots for Cardiology:
    * Doctor: Dr. Sarah Rao | Slot: 2026-09-10 10:00 | Match Score: 4.86 (SlotID:
SLOT_101)
    * Doctor: Dr. Sarah Rao | Slot: 2026-09-10 11:00 | Match Score: 4.86 (SlotID:
SLOT_102)
    * Doctor: Dr. John Doe | Slot: 2026-09-10 10:30 | Match Score: 4.14 (SlotID:
SLOT_201)

[+] Patient selecting optimal recommendation: Slot SLOT_101
[AUDIT LOG - 2026-09-03 03:32:35] Event: 'AppointmentBookedEvent' | Context:
{'appointment_id': '4c8e7f12', 'patient_id': 'PATIENT_8891', 'doctor_id': 'DOC_01',
'slot_id': 'SLOT_101', 'status': 'CONFIRMED'}
[+] Reservation Confirmed. Reference ID: 4c8e7f12

[+] Testing Race Condition Guard (simultaneous booking of reserved slot):
[-] Concurrency Reject: Slot SLOT_101 is no longer available.
[+] Concurrency control succeeded: double-booking prevented.
=====
==
```

18. Reflection on Design Decisions, Challenges, and SDG Alignment

18.1 Architectural Trade-offs & Decisions

- **Microservices vs. Modular Monolith:** A distributed microservice architecture was chosen over a monolith. While distributed services introduce network overhead and eventual consistency complexity, they allow the compute-heavy AI recommendation engine to scale independently on GPU clusters during morning peak booking hours.
- **Polyglot Data Storage Strategy:** PostgreSQL was chosen for appointment scheduling to guarantee ACID transactions and prevent double bookings, while AWS DocumentDB (MongoDB) was selected for semi-structured clinical SOAP notes and dynamic lab result schemas.
- **Optimistic vs. Pessimistic Locking:** Optimistic concurrency control with short-lived 10-minute Redis distributed leases was selected over pessimistic row locks. This maximizes system throughput while preventing stale bookings and race conditions.

18.2 Architectural Challenges & Solutions

- **Distributed State Synchronization:** Managing state transitions across appointment bookings, pharmacy dispensing, and lab orders without two-phase commit bottlenecks was challenging. This was resolved using the Saga Pattern (Choreography via Kafka), allowing each service to execute local transactions and emit compensating events upon failure.
- **Cold-Start Bias in Recommendations:** Newly onboarded physicians lacked historical punctuality data, skewing recommendation rankings. A fallback strategy based on verified credentials, geographic proximity, and availability density was implemented until sufficient ratings are collected.

18.3 Alignment with UN Sustainable Development Goals (SDGs)

- **SDG 3: Good Health and Well-Being:** AI-SHAPS improves healthcare access by matching patients with appropriate medical specialists and reducing waiting times through coordinated scheduling.
- **SDG 9: Industry, Innovation, and Infrastructure:** The cloud-native, model-driven architecture provides a resilient, modern digital foundation for healthcare facilities.
- **SDG 11: Sustainable Cities and Communities:** Centralized scheduling, electronic prescriptions, and digital lab reports reduce unnecessary physical hospital visits, helping lower urban traffic congestion and paper resource consumption.

